CHAPTER 07

Design A Key-value Store

A distributed, highly available, partition-tolerant store like Dynamo / Cassandra / Riak.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a key-value store that supports `put(key, value)` and `get(key)`. Small values (< 10 KB), big data, high availability, high scalability, automatic scaling, tunable consistency, low latency. The deep part of the question is how to build it as a **distributed** system and navigate the **CAP theorem**.

Why interviewers ask it: Brings together partitioning, replication, quorum consistency, conflict resolution, failure detection, repair and the read/write path.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Size of a key-value pair?	Small: < 10 KB.
Data volume?	Big data: far beyond a single machine.
Availability vs consistency?	High availability; tunable (eventual by default).
Scaling?	Automatic add/remove of servers based on traffic.
Latency?	Low (single-digit ms).

3. Requirements

Functional

- `put(key, value)` and `get(key)`.
- Overwrite existing keys; delete is a special marker (tombstone).

Non-functional

- Highly available: respond even during failures.
- Highly scalable: support a large dataset by adding nodes.
- Tunable consistency (N, W, R).
- Low latency; automatic scale-out; fault tolerant across data centers.

4. Back-of-the-envelope estimation

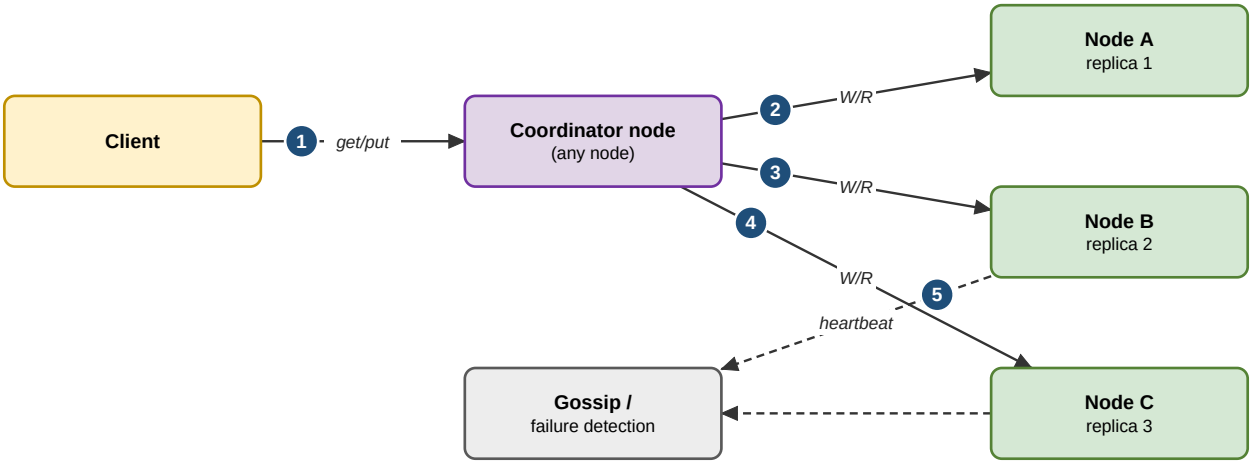
- Assume 1 B keys x 1 KB = 1 TB logical; replication factor 3 -> 3 TB raw. With 1 TB per node (leaving headroom) -> about 10 nodes minimum, plan 30+ for throughput and growth.
- At 100 K ops/s peak with ~5-10 K ops/s per node (disk bound) -> at least 10-20 nodes.

5. API / data model

```
put(key, value)          # value is an opaque blob; metadata may carry a version/vector clock
get(key) -> value | [conflicting values + context]
Config knobs: N (replicas), W (write quorum), R (read quorum)
```

6. High-level design

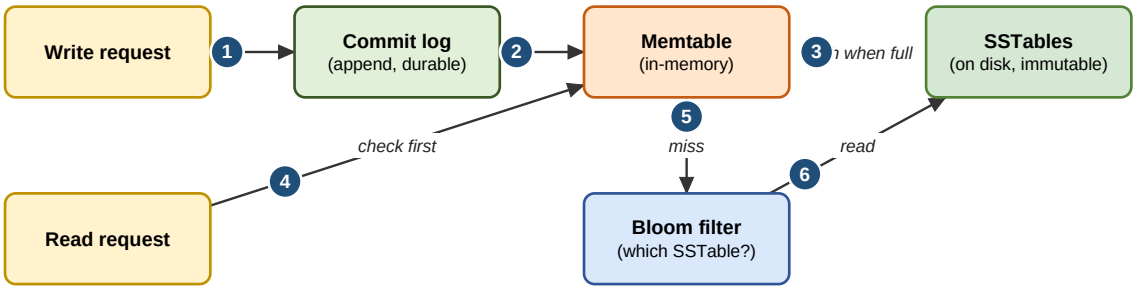
Coordinator-based cluster on a hash ring



Any node can coordinate a request. It hashes the key to the ring, forwards to N replicas, and answers when W (write) or R (read) acknowledge.

Step	What happens
1	The client sends `put` or `get` to **any node**; that node becomes the **coordinator** for this request.
2	The coordinator hashes the key onto the ring and forwards to the **N replica nodes** (the next N distinct physical servers clockwise). Node A is replica 1.
3	Same request goes to replica 2. The coordinator waits only for **W acks** (write) or **R replies** (read), not for all N.
4	Replica 3. With N=3, W=2, R=2, any read overlaps at least one up-to-date replica (W+R>N), giving strong consistency.
5	Nodes exchange **gossip heartbeats**; a node whose counter stops increasing is marked down, and the coordinator uses a sloppy quorum with hinted handoff meanwhile.

Write path (LSM tree) and read path



Step	What happens
1	A write is first appended to the **commit log** on disk, which gives durability (sequential I/O is fast).
2	Then it is applied to the in-memory **memtable**, so reads of recent data are fast.
3	When the memtable is full it is flushed to an immutable on-disk **SSTable**; background compaction merges tables and drops tombstones.
4	A read checks the memtable first.
5	On a miss the **Bloom filter** says which SSTables might contain the key (no false negatives), avoiding useless disk reads.
6	Read the candidate SSTables and return the newest version (reconcile using version or vector clock).

7. Deep dive

CAP theorem

- A distributed system can provide only two of **Consistency, Availability, Partition tolerance**. Partitions are unavoidable, so choose CP or AP.
- **CP** (e.g. bank balances, HBase, MongoDB-default): during a partition refuse to answer from stale replicas -> unavailable.
- **AP** (Dynamo, Cassandra): keep serving, accept stale data, reconcile later (eventual consistency). We design AP with tunable consistency.

Data partitioning

- Use **consistent hashing** (chapter 06) to spread data; virtual nodes balance load and allow heterogeneity; scaling moves minimal data.

Data replication

- Replicate each key to the next **N distinct physical servers** clockwise on the ring (across racks/data centers for safety).

Consistency: quorum (N, W, R)

- N = replicas; W = acks needed for a write; R = replicas that must answer a read.
- **W + R > N -> strong consistency** (read and write sets overlap). R = 1, W = N: fast reads. W = 1, R = N: fast writes. W = 1, R = 1: fastest, weak/eventual.
- Common: N = 3, W = 2, R = 2. The coordinator acts as a proxy between client and replicas.
- Models: strong, weak, **eventual** (Dynamo/Cassandra default).

Inconsistency resolution: versioning with vector clocks

- Concurrent writes to different replicas create conflicts. Replicas are immutable-versioned; the client or system reconciles.
- **Vector clock:** [server, counter] pairs attached to a value. Version X is an ancestor of Y if every counter in X <= Y. If neither dominates -> conflict.
- Downsides: adds client complexity; vector clocks can grow (truncate by a threshold). Alternative: last-write-wins (simple, may lose data), CRDTs.

Failure detection

- Needs more than one node saying a node is down. **Gossip protocol:** each node keeps a membership list with heartbeat counters, periodically sends heartbeats to random nodes; if a counter has not increased for a threshold, the node is marked offline.

Handling temporary failures: sloppy quorum + hinted handoff

- Strict quorum may block. **Sloppy quorum:** use the first W healthy nodes on the ring rather than the designated owners.
- A stand-in node stores the write with a **hint**; when the owner returns, it hands the data back.

Handling permanent failures: anti-entropy with Merkle trees

- Compare replicas efficiently: hash key ranges into buckets, hash buckets into a tree. Compare roots; descend only into differing branches.
- Only differing buckets are synchronised, so repair bandwidth is proportional to the differences, not the data size.

Data center outage

- Replicate across multiple data centers so users can be served from another DC after a regional failure.

Write path (Cassandra-style)

- 1) Append to the **commit log** for durability. 2) Write to the in-memory **memtable**. 3) When full, flush to an immutable on-disk **SSTable**. 4) Background compaction merges SSTables and drops tombstones.

Read path

- Check memtable -> if miss, consult **Bloom filter** to find candidate SSTables -> read SSTables -> return the newest value (merge by version).

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Consistency level	Strong (W+R>N) / Eventual (W=1,R=1)	Tune per use case; carts and counters tolerate eventual, balances do not.
Conflict resolution	Vector clocks / Last-write-wins / CRDTs	Vector clocks or CRDTs for no data loss; LWW if simplicity beats rare lost updates.
Storage engine	LSM tree (write-optimised) / B-tree (read-optimised)	LSM for write-heavy KV; B-tree for read-heavy with range scans.
Coordinator	Client-side routing / Server-side coordinator	Coordinator keeps clients thin; client-side cuts a hop.

9. Scaling, reliability, failure modes

- Decentralised: no single point of failure; every node has the same responsibilities.
- Add a node -> it takes vnodes -> streams data from neighbours -> starts serving.
- Use tombstones for deletes (so replicas converge) and compaction to reclaim space.

10. Likely follow-up questions

- What exactly happens if two clients write the same key concurrently to different replicas?
- How do you size N, W, R for a shopping cart vs an account balance?
- How would you add range scans or secondary indexes?
- How do you handle hot keys and large values?

11. Common mistakes

- Saying 'CA system' - partition tolerance is not optional in a distributed system.
- Ignoring failure detection and repair: only describing the happy path.
- Not explaining how conflicts are detected (vector clocks) vs silently using LWW.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	4 min	"I'll assume small values under 10 KB, a very large dataset, high availability and low latency, with tunable consistency rather than a single fixed mode. I know a distributed system must tolerate partitions, so the real choice is CP versus AP; for a general-purpose store I'll default to AP with tunable quorums. Is that okay?"
2. High-level design	10 min	"I partition data with consistent hashing and virtual nodes, replicate each key to N nodes, and let any node act as a coordinator. The coordinator forwards to the replicas and waits for W acks on writes or R replies on reads. Locally each node uses an LSM tree: commit log, memtable, SSTables."
3. Deep dive	25 min	"For consistency, W + R > N gives strong consistency, while W=R=1 is fastest but eventual. Concurrent writes produce conflicts, which I detect with vector clocks. For failures: gossip for detection, sloppy quorum with hinted handoff for temporary failures, and Merkle trees for anti-entropy after permanent ones. Reads go memtable then Bloom filter then SSTables."
4. Wrap up	4 min	"Summary: ring partitioning, N-way replication, tunable quorums, vector clocks, gossip, hinted handoff and Merkle trees. Open issues are hot keys, large values, and secondary indexes; I'd monitor replica lag and compaction backlog."

13. 60-second recap / practice checklist

- Ring partitioning (consistent hashing + vnodes) + N-way replication + tunable quorum.
- AP by default: gossip, sloppy quorum + hinted handoff, vector clocks, Merkle-tree anti-entropy.
- Write: commit log -> memtable -> SSTable. Read: memtable -> Bloom filter -> SSTable.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

- Dynamo paper (SOSP 2007)
- Bigtable paper
- Apache Cassandra architecture
- GeeksforGeeks: CAP theorem